

Assessing ChatGPT's Code Generation Capabilities with Short vs Long Context Programming Problems

Uddip Acharjee Shuvo*
Institute of Information Technology
University of Dhaka, Bangladesh
Dhaka, Bangladesh
uddip.acharjee.shuvo@gmail.com

Nirvar Roy Vaskar
American International University - Bangladesh
Dhaka, Bangladesh
nirvarroy@hotmail.com

Sajib Acharjee Dip[†]
Computer Science and Engineering
Virginia Tech, USA
Blacksburg, USA
sajibacharjeedip@vt.edu

A. B. M. Alim Al Islam[‡]
Computer Science and Engineering
Bangladesh University of Engineering and Technology
Dhaka, Bangladesh
razi_bd@yahoo.com

Abstract

This study assesses the code generation capabilities of ChatGPT using competitive programming problems from platforms such as LeetCode, HackerRank, and UVa Online Judge. In a novel approach, we contrast ChatGPT's performance on concise problems from LeetCode against more complex, narrative-driven problems from Codeforces. Our results reveal significant challenges in addressing the intricate narrative structures of Codeforces, with difficulties in problem recognition and strategic planning in extended contexts. While initial code accuracy for LeetCode problems stands at 72%, it drops to 31% for complex Codeforces problems using Python. Additionally, we explore the impact of targeted instructions aimed at enhancing performance, which increased LeetCode accuracy to 73.53% but saw a decrease in Codeforces performance to 29%. Our analysis further extends across multiple programming languages, examining if iterative prompting and specific feedback can enhance code precision and efficiency. We also delve into ChatGPT's performance on challenging problems and those released post its training period. This research provides insights into the strengths and weaknesses of AI in code generation and lays groundwork for future developments in AI-driven coding tools.

CCS Concepts

• **Computing methodologies** → **Artificial intelligence**; **Natural language processing**; • **Software and its engineering** → **General programming languages**; • **Applied computing** → **Document management and text processing**; • **General and reference** → **Empirical studies**.

*Corresponding Author. Both authors contributed equally to this research.

[†]Both authors contributed equally to this research.

[‡]Corresponding Author



This work is licensed under a Creative Commons Attribution International 4.0 License.

NSysS '24, December 19–21, 2024, Khulna, Bangladesh
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1158-9/24/12
<https://doi.org/10.1145/3704522.3704535>

Keywords

Competitive Programming, Code Generation, AI-Assisted Programming, Language Models, ChatGPT, Prompt Engineering, Performance Evaluation

ACM Reference Format:

Uddip Acharjee Shuvo, Sajib Acharjee Dip, Nirvar Roy Vaskar, and A. B. M. Alim Al Islam. 2024. Assessing ChatGPT's Code Generation Capabilities with Short vs Long Context Programming Problems. In *11th International Conference on Networking, Systems, and Security (NSysS '24)*, December 19–21, 2024, Khulna, Bangladesh. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3704522.3704535>

1 Introduction

Large language models (LLMs) have demonstrated remarkable capabilities across a wide range of domains, showcasing their potential in tasks such as reasoning [50], summarization [36, 53, 54], contextual understanding [15], biological sequence analysis [8, 9, 17], medical diagnosis [16, 55], and software development. Among these applications, their ability to assist in code generation and software engineering stands out as particularly transformative. The growing development of large language models (LLMs) has profoundly influenced the field of automated code generation. Models like GPT-4 and its successors have demonstrated unprecedented capabilities, offering a stark contrast to traditional deep learning approaches that often struggled with complex and varied programming tasks. Prior research highlighted the potential of deep learning models to generate functional code from natural language descriptions, yet these models frequently faltered with generalizing to unseen problems and managing long-context dependencies [2, 22, 26, 42, 51].

Recent advancements have positioned LLMs at the forefront of this research area. Unlike their predecessors, LLMs leverage extensive training across diverse datasets, enabling them to produce syntactically and semantically correct code across various programming languages [7, 13, 29]. However, studies have revealed inconsistencies in the performance of these models, particularly in terms of execution times and efficiency when dealing with complex problems [18, 43, 49]. In addition, the integration of LLMs into educational settings has opened new avenues for their application, suggesting potential benefits in programming courses. Although

these models can rival proprietary systems like ChatGPT (GPT-4 as the backbone) [10] in performance, challenges in the quality and consistency of the feedback they provide remain significant hurdles. Despite these advancements, LLMs are not without their challenges. A pivotal limitation is their ability to effectively handle long-context problems. While excelling in tasks involving short to moderate complexities, their performance notably declines with tasks that require a deep understanding of extended narratives or complex logical dependencies [28, 33]. [4] discusses the limitations of LLMs in handling long contexts in code generation, specifically their ability to maintain coherence over extended text spans. Building on these insights, our research aims to explore several critical areas within this evolving landscape:

- We investigate how LLMs handle code generation tasks with varying context lengths, which also assesses the trade-offs involved.
- We also examine the ability of LLMs to tackle complex and logic-intensive problems, which also highlighted potential difficulties for traditional AI models.
- We evaluate the generalization capabilities of ChatGPT, focusing on their adaptability to new and challenging scenarios.
- Furthermore, we analyze the influence of various programming languages and evaluate the impact of prompt design on the efficiency and quality of the generated code.

By curating a comprehensive dataset from platforms such as LeetCode[27] and Codeforces[12], this paper provides a detailed empirical analysis that enriches our understanding of LLMs' code generation capabilities. Our findings contribute to the broader discourse on the application of tools like ChatGPT in software engineering, identifying key strengths and ongoing challenges. This research aims to guide the development of more robust and efficient LLM-based tools, enhancing their practical utility in diverse programming tasks.

2 Related Works

The literature on Large Language Models (LLMs) like ChatGPT in software engineering is extensive and varied, reflecting different aspects of the field, which is shown with their key findings in Table 1. It can be grouped into several key themes:

Code Generation and Maintenance: Research in this area focuses on how LLMs generate and maintain code, evaluating the accuracy and efficiency of the code with the objective of reducing coding time and enhancing code reliability [5, 19, 23, 25, 45, 46, 52].

Software Development Assistance: Studies explore LLM support throughout the software development lifecycle, including planning, testing, and debugging, focusing on how they improve productivity through consultations, test case generation, and code optimization [6, 37, 39].

Educational and Learning Applications: This theme examines the role of LLMs in software engineering education, such as helping learners grasp complex concepts and providing real-time feedback [20, 30, 35, 44].

User Experience and Interaction: These studies investigate how software engineers use LLMs in practice, analyzing user satisfaction, tool adaptability, and trust in tool output [48].

Our research advances the field by analyzing shortcomings identified in previous studies on code generation ability of large-language models (LLMs) [11, 13, 14, 24, 25, 29, 31, 34, 38, 40, 41]. We uniquely assess how LLMs handle code generation with varying context lengths and complexity by using a comprehensive dataset from platforms like LeetCode and Codeforces. This study not only examines LLMs' capability to solve complex, logic-intensive problems but also their adaptability to new and unforeseen challenges. Moreover, we explore the impact of different programming languages and prompt designs on the efficiency of the generated code. Our findings help in understanding LLMs' operational strengths and limitations in real-world scenarios, guiding the development of more effective LLM-based tools for diverse software engineering tasks.

3 Methodology

3.1 Data Source Selection for Short and Long Context Problems

For our study assessing ChatGPT's code generation capabilities across varying context lengths, we surveyed several prominent online programming platforms such as Hackerrank[21], LeetCode[27], UVa[47], and Codeforces[12]. Each platform offers unique approaches to presenting programming problems: LeetCode provides concise problem descriptions with clear constraints and multiple examples; Codeforces presents problems within a narrative, detailing inputs, outputs, and constraints in a story-like format and sometimes includes additional notes. UVa's descriptions are intermediate, neither as concise as LeetCode nor as narrative-driven as Codeforces, usually encompassing all details within a single paragraph along with examples. Hackerrank closely resembles LeetCode but places constraints prior to sample inputs and outputs. Given LeetCode's high popularity and precise problem descriptions, we selected it to represent platforms with short and exact contexts, while Codeforces was chosen to exemplify those with extended narrative contexts.

3.2 Data Collection and Preparation

To create a diverse and representative dataset, we carefully compiled programming problems from a variety of categories and difficulty levels. Specifically, we selected 102 problems from LeetCode, evenly distributed among three difficulty categories: easy, medium, and hard, with each category containing 34 problems. From Codeforces, we curated 150 problems, also evenly split across three analogous categories shown in Fig. 3C. However, since Codeforces does not explicitly label problems as easy, medium, or hard, we classified them based on their difficulty scores, which range from 800 to 3500, into three bins: 800-1200 (easy), 1200-1800 (medium), and 1800-3500 (hard).

Given that GPT-4, which powers ChatGPT, was trained on data up to September 30, 2021, we divided our dataset into two segments: problems published before and after this date. This division helps evaluate how well GPT-4 handles previously unseen problems, testing its ability to generalize beyond its training data. The dataset spans a broad spectrum of programming topics, including arrays, sorting, strings, greedy algorithms, hashing, binary search trees, dynamic programming, constructive algorithms, and mathematical challenges. We also incorporated less common topics such as

Table 1: Qualitative Comparison of Recent Studies on LLMs for Code Generation

Study & Year	Methodology & Tasks	Dataset Used	Metrics & Key Findings
Jin et al., 2024 [24]	Empirical analysis using the DevGPT dataset to evaluate interaction dynamics and code utility in development.	DevGPT dataset from GitHub interactions, 17,913 prompts and responses.	Analyzes conversation structures and code usability; highlights that ChatGPT's output is frequently not production-ready, necessitating improvements for real-world application.
Siddiq et al., 2024 [40]	Analysis of 3,566 prompts from 9 benchmarks to assess quality issues affecting model performance.	Benchmarks for Python and Java code generation, focusing on contextual dependency and prompt clarity.	Found primary issues with prompt quality affecting performance; highlighted spelling, grammar errors, and lack of proper documentation style. Also noted potential data contamination in benchmarks.
Liu et al., 2023 [29]	Evaluated LLM code generation correctness using EvalPlus (extended benchmarks with augmented test cases).	EvalPlus on HUMANEVAL+, extensive testing across 26 LLMs.	Revealed significant performance drops and misrankings, demonstrating test insufficiency and the need for benchmark improvement.
Miah & Zhu, 2024 [31]	Proposes a user-centric evaluation method for LLMs in code generation, focusing on usability and performance from user interactions.	Case study on ChatGPT for R programming, including enhanced test cases and usability metrics.	Demonstrates ChatGPT's effectiveness yet identifies challenges in hard tasks, with a focus on improving usability metrics based on user interaction.
Ghosh Paul et al., 2024 [34]	Critically reviews benchmarks and metrics used in the evaluation of LLMs for code generation, highlighting inconsistencies and research gaps.	Reviews a broad range of benchmarks and metrics across various LLM platforms.	Discusses the inadequacies of current metrics and benchmarks in reflecting true user experience, suggesting directions for future research.
Ságoti et al., 2024 [38]	Evaluates ChatGPT and Copilot for code synthesis using a new methodology on 25 programming tasks.	Functional and non-functional code qualities assessed, showing ChatGPT outperformed Copilot.	Demonstrates methodology effectiveness by comparing code quality, developer acceptance, and model choice considerations.
Cotroneo et al., 2024 [14]	Proposes ACCA to automate the evaluation of AI-generated security code against human evaluation.	Uses symbolic execution; assesses four NMT models.	Finds high correlation with human evaluation, outperforming baselines; significantly faster than manual methods.
Bucaioni et al., 2024 [11]	Examines ChatGPT's ability to write source code in C++ and Java compared to human programmers.	Uses LeetCode problems as benchmarks; assesses code correctness, runtime efficiency, and memory usage.	Finds ChatGPT effective for easy and medium problems, but less so for complex tasks, with suboptimal runtime and memory performance compared to humans.
Andino Coello et al., 2024 [13]	Evaluates the effectiveness of ChatGPT and other LLMs in programming tasks using the MBPP dataset.	Compares GPT-3.5 and GPT-4 with Google's Bard and Anthropic's Claude.	Finds ChatGPT models to perform well in coding tasks, providing insights for developers and researchers in AI-assisted programming. Highlights challenges and limitations of LLMs in programming.
Siddiq et al., 2024 [41]	Examines quality issues in ChatGPT-generated code using DevGPT dataset across Python & Java platforms.	Analyzes code quality, merge practices, and developer utilization of ChatGPT-generated code in software development.	Reveals significant modifications before merging ChatGPT code and highlights diverse use cases of ChatGPT in software development stages. Identifies common code quality issues like security vulnerabilities and improper documentation.
Khojah et al., 2024 [25]	Observational study of 24 software engineers using ChatGPT over one week for various software engineering tasks.	Explores how software engineers utilize ChatGPT beyond code generation, including planning, learning, and guidance.	Finds ChatGPT primarily used for high-level guidance rather than direct code manipulation. Introduces a framework analyzing the influence of user interaction purposes and internal/external factors on the perceived usefulness and trust of ChatGPT.
Ni et al., 2024 [32]	Evaluates language-to-code tasks, focusing on scaling, data mixtures, and instruction tuning with standardized metrics.	Evaluates 56 LLMs from 13 organizations across semantic parsing, math reasoning, and Python programming benchmarks.	Highlights that larger, instruction-tuned models excel in reasoning and few-shot tasks, while code-specific models achieve high efficiency; the framework emphasizes execution accuracy and supports reproducibility.

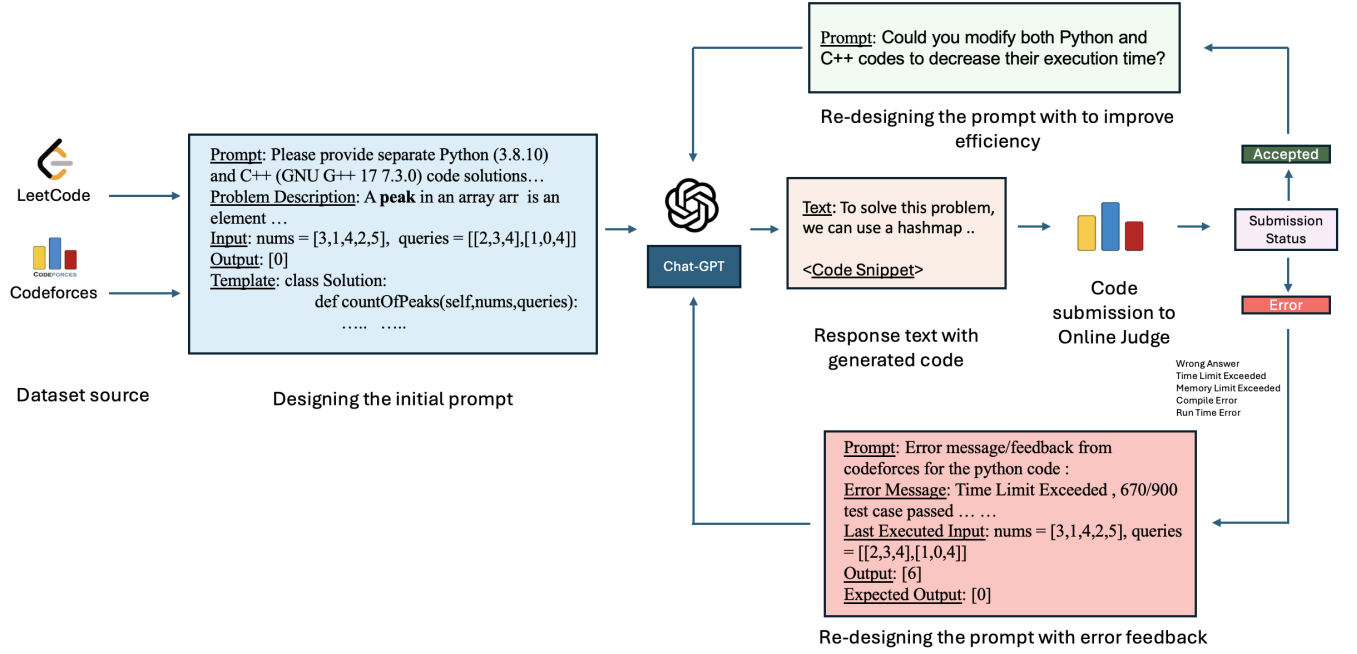


Figure 1: Experimental Workflow: The process begins with the collection of data from the corresponding website, which includes problem descriptions, input/output specifications, templates, and constraints. This data guides ChatGPT in generating code, which is subsequently evaluated by an online judge. If the initial solution is accepted, the next prompt focuses on enhancing runtime efficiency. Conversely, if the initial submission is rejected, the subsequent prompt is crafted to address the specific errors highlighted by the online judge, aiming to resolve any bugs in the code.

concurrency, prefix sums, enumeration, and databases to ensure comprehensive coverage and robustness in our analysis.

3.3 Prompt Designing and Experimental Workflow

In our study, we employed a standardized prompt engineering technique to guide ChatGPT-4 in generating C++ (GNU G++ 17 7.3.0) and Python (version 3.8.10) code. This approach minimizes compatibility issues by specifying exact software versions. Each prompt incorporated a default problem description from online coding platforms such as LeetCode and Codeforces, along with sample inputs and outputs to facilitate few-shot learning shown in Fig. 3A.

The prompts included constraints and explanations to ensure that the solutions generated by ChatGPT adhered to the given requirements. After submitting the initial prompt, ChatGPT-4's output comprised explanatory text and code snippets in both C++ and Python, as detailed in Fig. 3B. We extracted these code snippets and submitted them to the respective platforms, specifying the programming language used for each submission.

The results, including whether the solutions were accepted, specific errors encountered, required time complexity, and other pertinent details, were meticulously documented. If a solution was accepted, a subsequent prompt was crafted to enhance code efficiency. Conversely, if the solution was rejected, we refined the

prompt to address the errors by incorporating feedback from the online judge, including failed test cases. This iterative process allowed us to better understand and rectify the bugs. The revised code was then resubmitted, and we recorded the changes in performance, noting any improvements or declines.

4 Evaluation

To rigorously assess the correctness, efficiency, and effectiveness of the feedback-learning process, we implemented a comprehensive set of evaluation metrics. For assessing correctness, we categorized outcomes into six statuses: Accepted (AC), Wrong Answer (WA), Time Limit Exceeded (TLE), Runtime Error (RTE), Memory Limit Exceeded (MLE), and Compile Error (CE). We calculated the percentage distribution of each status across various problem sub-categories to measure performance accurately. Efficiency was gauged by comparing execution times across runs, highlighting feedback's impact on performance improvement. The rate for each evaluation status can be generalized as follows, where Status represents any of the specific outcomes: AC, WA, TLE, RTE, MLE, or CE.

$$\text{Rate of Status (\%)} = \left(\frac{\text{Total number of problems with Status}}{\text{Total problems}} \right) \times 100\%$$

Table 2: Performance of ChatGPT Generated Code in Python Using Initial Prompt

Period	Complexity	LeetCode						Codeforces					
		AC	WA	TLE	RTE	MLE	CE	AC	WA	TLE	RTE	MLE	CE
Before	Easy	100%	0%	0%	0%	0%	0%	84%	12%	0%	0%	0%	4%
	Medium	100%	0%	0%	0%	0%	0%	48%	44%	4%	4%	0%	0%
	Hard	88.24%	5.88%	0%	5.88%	0%	0%	4%	80%	8%	4%	0%	4%
	Total	96%	2%	0%	2%	0%	0%	45%	45%	4%	3%	0%	3%
After	Easy	88.24%	5.88%	0%	5.88%	0%	0%	20%	80%	0%	0%	0%	0%
	Medium	29.41%	52.94%	11.76%	0%	5.88%	0%	28%	60%	8%	4%	0%	0%
	Hard	23.53%	41.18%	29.41%	0%	0%	5.88%	0%	92%	4%	4%	0%	0%
	Total	47.06%	33.33%	14%	1.96%	2%	2%	16%	77%	4%	3%	0%	0%
Total		72%	18%	7%	2%	1%	1%	31%	61%	4%	3%	0%	1%

Table 3: Performance of ChatGPT generated code in C++ using Initial Prompt

Period	Difficulty	LeetCode						Codeforces					
		AC	WA	TLE	RTE	MLE	CE	AC	WA	TLE	RTE	MLE	CE
Before	Easy	100%	0%	0%	0%	0%	0%	88%	12%	0%	0%	0%	0%
	Medium	94.11%	0%	0%	5.88%	0%	0%	44%	40%	8%	0%	0%	8%
	Hard	82.35%	0%	0%	5.88%	0%	11.76%	4%	68%	8%	4%	0%	16%
	Total	92%	0%	0%	4%	0%	4%	45%	40%	5%	1%	0%	8%
After	Easy	88.24%	5.88%	0%	0%	0%	5.88%	20%	76%	0%	0%	0%	4%
	Medium	29.41%	52.94%	11.76%	0%	5.88%	0%	20%	64%	4%	4%	0%	8%
	Hard	17.65%	35.29%	35.29%	5.88%	0%	5.88%	0%	80%	4%	4%	0%	12%
	Total	45.10%	13%	7%	3%	1%	4%	13%	73%	3%	3%	0%	8%
Total		69%	7%	3%	4%	0%	4%	29%	57%	4%	2%	0%	8%

Table 4: Performance of ChatGPT generated code in Python using Second Run Prompt

Period	Difficulty	LeetCode						Codeforces					
		AC	WA	TLE	RTE	MLE	CE	AC	WA	TLE	RTE	MLE	CE
Before	Easy	94.11%	5.88%	0%	0%	0%	0%	68%	24%	0%	4%	0%	4%
	Medium	94.11%	0%	0%	5.88%	0%	0%	40%	60%	0%	0%	0%	0%
	Hard	82.35%	11.76%	0%	5.88%	0%	0%	4%	88%	4%	4%	0%	0%
	Total	90.19%	5.88%	0%	4%	0%	0%	37%	57%	1%	3%	0%	1%
After	Easy	88.24%	5.88%	5.88%	0%	0%	0%	24%	76%	0%	0%	0%	0%
	Medium	58.82%	35.29%	0%	5.88%	0%	0%	36%	64%	0%	0%	0%	0%
	Hard	23.53%	47.06%	17.64%	5.88%	0%	0%	0%	84%	0%	16%	0%	0%
	Total	56.86%	29.41%	7.84%	4%	0%	0%	20%	75%	0%	5%	0%	0%
Total		73.53%	17.65%	4%	4%	0%	0%	29%	66%	1%	4%	0%	1%

5 Results and Discussions

5.1 Evaluating the Correctness of Code Generated by ChatGPT

We conducted a thorough performance evaluation using our curated dataset, comparing the effectiveness of ChatGPT in handling

problems from Codeforces and LeetCode across six evaluation metrics. Our findings reveal that ChatGPT demonstrates a markedly higher proficiency with the concise and well-structured problems found on LeetCode, achieving an overall accuracy of 72% for Python code on the first attempt, compared to just 31% for Codeforces. Additionally, the incidence of wrong answers increased from 18% to 31% when transitioning from short to long-context problems, as

Table 5: Performance of ChatGPT generated code in C++ using Second Run Prompt

Period	Difficulty	LeetCode						Codeforces					
		AC	WA	TLE	RTE	MLE	CE	AC	WA	TLE	RTE	MLE	CE
Before	Easy	100%	0%	0%	0%	0%	0%	76%	20%	0%	0%	0%	4%
	Medium	94.11%	5.88%	0%	0%	0%	0%	56%	44%	0%	0%	0%	0%
	Hard	70.59%	17.64%	0%	5.88%	0%	5.88%	4%	92%	4%	0%	0%	0%
	Total	88%	8%	0%	2%	0%	2%	45%	52%	1%	0%	0%	1%
After	Easy	94.11%	5.88%	0%	0%	0%	0%	24%	76%	0%	0%	0%	0%
	Medium	58.82%	35.29%	0%	0%	0%	5.88%	36%	60%	0%	0%	0%	4%
	Hard	29.41%	41.18%	23.53%	0%	0%	5.88%	0%	80%	4%	0%	0%	0%
	Total	60.78%	27.45%	8%	0%	0%	4%	20%	72%	1%	0%	0%	1%
Total		75%	18%	4%	1%	0%	3%	33%	62%	1%	0%	0%	1%

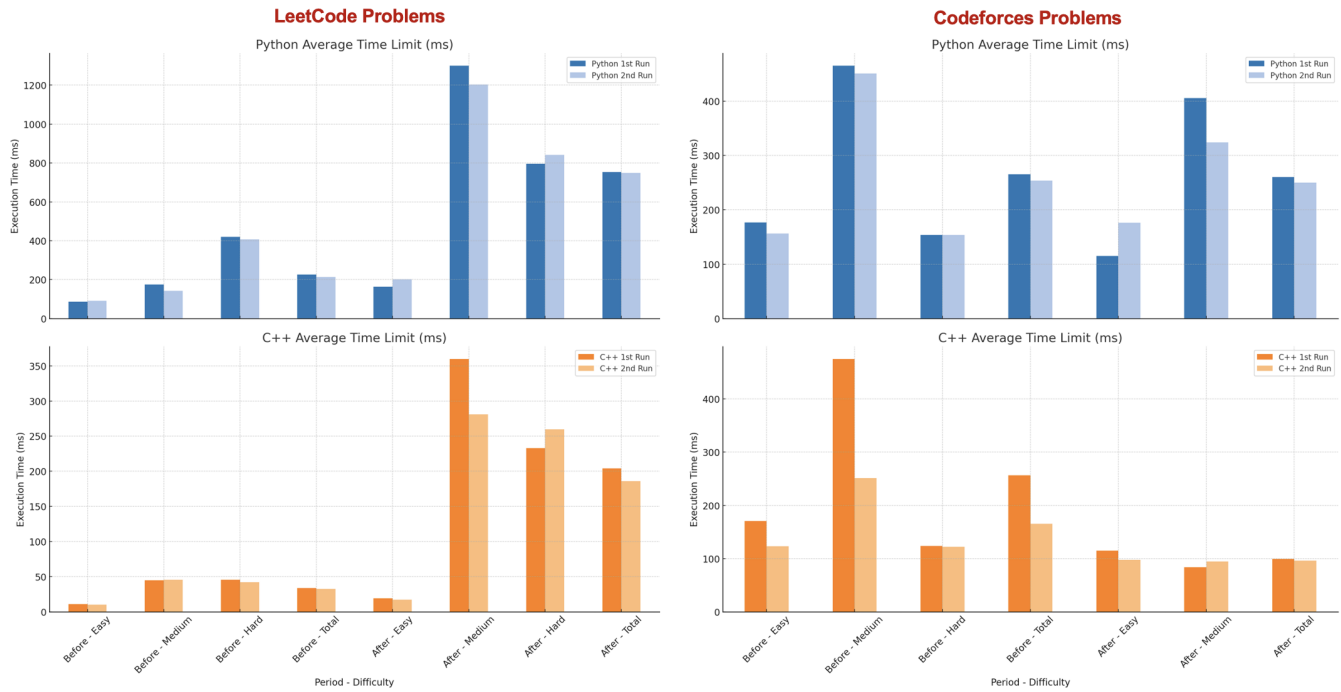


Figure 2: Time Limit Comparison: This figure illustrates the differences in execution times between initial and redesigned prompts for solving problems on LeetCode and Codeforces. Execution times are shown for Python (top panels) and C++ (bottom panels) programming languages, with LeetCode results on the left and Codeforces results on the right. The data indicate that C++ generally requires less execution time than Python. Additionally, a redesign of the prompts consistently improved runtime performance, with noted exceptions in medium and hard problems post-September 2021.

detailed in Table 2. The error rates for other categories also showed similar trends.

This disparity in performance can be attributed primarily to the differing presentation styles of the platforms. LeetCode’s direct and unambiguous problem statements align closely with ChatGPT’s training, facilitating more accurate code generation. In contrast, Codeforces employs a narrative approach that intertwines critical problem details within elaborate stories, posing significant challenges for the AI. Parsing these complex narratives effectively to

extract pertinent information necessitates more sophisticated natural language processing techniques, highlighting a crucial area for further enhancement of AI capabilities in code generation.

5.2 Variability in Code Generation Performance Across Different Programming Languages

To ensure a thorough analysis, we expanded our study to include C++ alongside Python, covering a broad spectrum of programming paradigms. Python, being high-level and interpreted, is ideal for

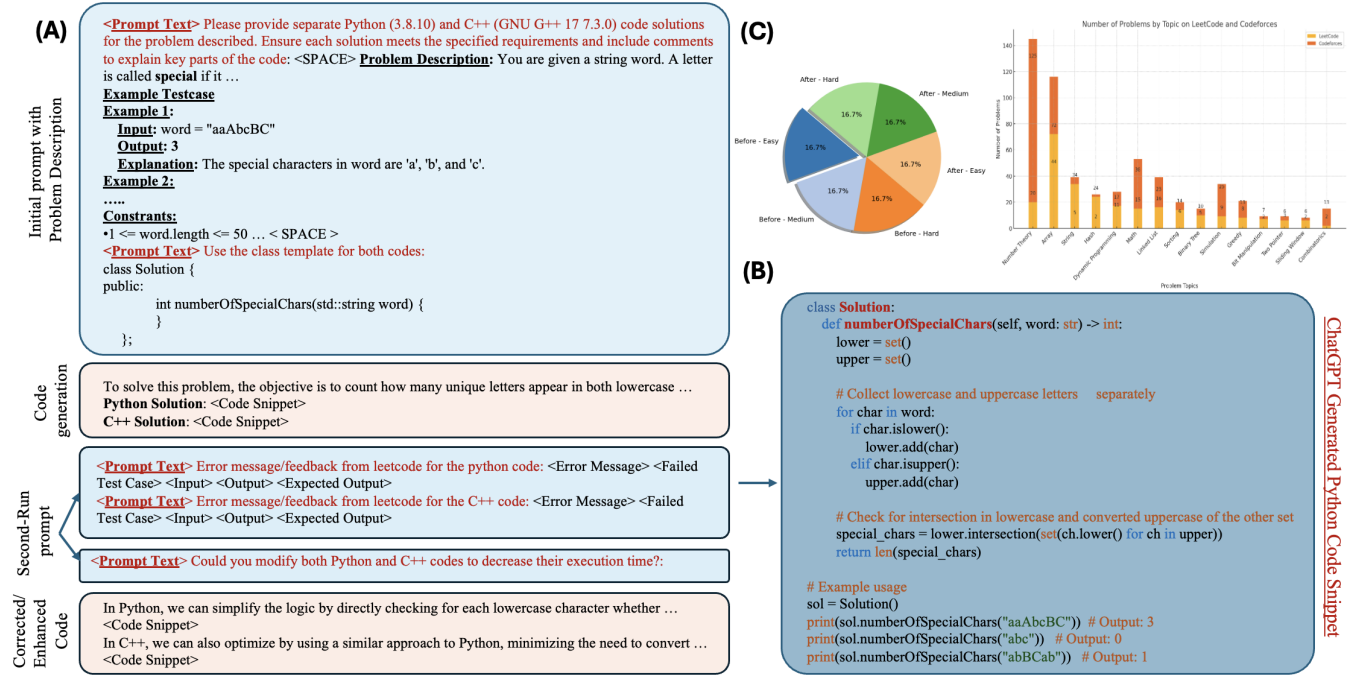


Figure 3: (A) Case study illustrating ChatGPT's problem-solving capabilities, featuring a two-round interaction with prompts and ChatGPT responses, including code enhancements and bug fixing. **(B)** Example of ChatGPT-generated code, annotated with comments and statements formatted for evaluation purposes. **(C)** Distribution of the dataset, categorizing problems into easy, medium, and hard difficulties, across time periods before and after September 2021, demonstrating balance across difficulty levels and temporal coverage with an overview of primary topics.

rapid development in web and data analytics, while C++, a low-level, compiled language, offers precise resource control, crucial for performance-critical software. This approach highlights how different programming languages affect code generation performance and applicability.

Our results indicate that Python generally outperformed C++ in initial tests. Specifically, the acceptance rates for Python were 72% for LeetCode and 31% for Codeforces, compared to 69% and 29% for C++, showing a decrease of 3% and 2%, respectively. Furthermore, C++ exhibited a higher rate of compiling errors, as documented in Table 3. In subsequent runs, performance trends reversed, highlighting variations in language adaptability over multiple iterations. This shift emphasizes the influence of language features on code generation success and error rates, setting the stage for further research into AI-driven adaptive code synthesis in varied programming contexts. Further details on this performance reversal are discussed in the next section.

5.3 Enhancing Code Generation Through Error Feedback and Prompt Redesign

Our study aimed to assess the effectiveness of ChatGPT's ability to integrate error feedback from programming platforms and improve its code generation through prompt redesign. Specifically, we focus on programs that were initially rejected and subsequently

re-evaluated with modifications suggested by ChatGPT. This iterative process, incorporating direct feedback from run-time and compilation errors, allowed us to gauge the adaptability and learning capabilities of the model. The results demonstrated notable improvements in most cases. For example, in LeetCode submissions, the acceptance rate for Python slightly increased from 72% to 73.53%, while for C++, it saw a more substantial rise from 69% to 75% and was reflected in Table 4. These enhancements underscore the benefits of incorporating specific, contextual feedback into the AI's learning loop, allowing it to refine its approach and output. Codeforces showed varied results; C++ acceptance improved, while Python's declined from 31% to 29%. This reflects GPT-4's potential limitations with complex narrative-driven problems requiring nuanced understanding. The selection of the data set and the topics significantly affected the outcomes, underscoring how model performance varies with the nature of the problem and feedback. These results emphasize the importance of targeted feedback and tailored interactions to improve AI code generation.

5.4 Impact of Targeted Prompts on the Time Complexity of ChatGPT-Generated Programs

In our study, we analyzed the ability of ChatGPT to enhance the efficiency of programming solutions that were initially accepted in the first run. Using targeted prompts aimed at optimizing the

time complexity, we observed a consistent reduction in the average execution time across both datasets before and after training, for both LeetCode and Codeforces. This demonstrates the potential of fine-tuned AI prompts to significantly improve the operational efficiency of the code. The results varied by subcategory, as shown in Fig. 2; for example, easy LeetCode problems before 2021 saw an increase in solution times from 86.24 milliseconds to 90.53 milliseconds in the second run, suggesting that smaller samples may not reflect broader trends and highlighting the complexity of AI-driven code optimization.

5.5 Challenges With Harder and Unseen Problems

Performance on harder problems significantly declined for both LeetCode and Codeforces, with difficulties exacerbated by Codeforces' lengthy narratives. Furthermore, ChatGPT struggled notably with unseen problems: while acceptance rates for known problems (before 2021) reached 96% on LeetCode and 45% on Codeforces, these dropped to just 47.06% and 16% for unseen problems in Python, as shown in Table 2. This trend was consistent for C++. Subsequent runs slightly narrowed the performance gap between seen and unseen problems, yet challenges persist in optimizing responses for unfamiliar queries.

5.6 Case Study: Repeated Tests of a Specific Prompt and the Resulting ChatGPT Code Outputs

We presented a case study of an easy category problem from LeetCode post-September 2021, titled "Count the Number of Special Characters I." This involved designing a prompt that incorporated the problem description, extracting the initial Python code generated by ChatGPT, refining the prompt based on initial outputs, and reevaluating the modified code as shown in Fig. 3A. The right panel of the figure displays the Python code from the first run, which was noted for its readability and cleanliness. The comments added by ChatGPT were generic yet effective in highlighting the subtasks involved.

6 Conclusion

Our study highlights several key insights into ChatGPT's performance across varying problem contexts and programming languages. While ChatGPT exhibits robust performance on well-structured, short-context problems from LeetCode, it faces considerable challenges with the less structured, narrative-rich problems from Codeforces. Although iterative prompting showed some potential in reducing the performance gap between known and unknown problems, significant challenges remain. The study's limitations include a relatively small dataset, though its diversity across problem types provides a valuable preliminary insight. Future research will expand on this work by incorporating a broader array of data, additional programming languages, and comparing performance against other advanced generative models such as Llama-70B[1] and Claude3[3]. This expanded approach will aim to further delineate the strengths and weaknesses of AI-driven code generation in a competitive programming context.

References

- [1] AI@Meta. 2024. Llama 3 Model Card. (2024). https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md
- [2] Miltiadis Allamanis, Earl T Barr, Premkumar Devanbu, and Charles Sutton. 2018. A survey of machine learning for big code and naturalness. *ACM Computing Surveys (CSUR)* 51, 4 (2018), 1–37.
- [3] AI Anthropic. 2024. The claude 3 model family: Opus, sonnet, haiku. *Claude-3 Model Card* 1 (2024).
- [4] Yannick Assogba and Donghao Ren. 2024. Evaluating Long Range Dependency Handling in Code Generation Models using Multi-Step Key Retrieval. *arXiv preprint arXiv:2407.21049* (2024).
- [5] Ben Athiwaratkun, Sanjay Krishna Gouda, Zijian Wang, Xiaopeng Li, Yuchen Tian, Ming Tan, Wasi Uddin Ahmad, Shiqi Wang, Qing Sun, Mingyue Shang, et al. 2022. Multi-lingual evaluation of code generation models. *arXiv preprint arXiv:2210.14868* (2022).
- [6] Somnath Banerjee, Avik Dutta, Sayan Layek, Amrui Sahoo, Sam Conrad Joyce, and Rima Hazra. 2023. Redefining Developer Assistance: Through Large Language Models in Software Ecosystem. *arXiv preprint arXiv:2312.05626* (2023).
- [7] Robin Beer, Alexander Feix, Tim Guttzeit, Tamara Muras, Vincent Müller, Maurice Rauscher, Florian Schäffler, and Welf Löwe. 2024. Examination of Code generated by Large Language Models. *arXiv:2408.16601 [cs.SE]* <https://arxiv.org/abs/2408.16601>
- [8] Gonzalo Benegas, Chengzhong Ye, Carlos Albors, Jianan Canal Li, and Yun S Song. 2024. Genomic language models: opportunities and challenges. *arXiv preprint arXiv:2407.11435* (2024).
- [9] Zhenyu Bi, Sajib Acharjee Dip, Daniel Hajjaligol, Sindhura Kommu, Hanwen Liu, Meng Lu, and Xuan Wang. 2024. AI for Biomedicine in the Era of Large Language Models. *arXiv preprint arXiv:2403.15673* (2024).
- [10] Tom B Brown. 2020. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165* (2020).
- [11] Alessio Bucaioni, Hampus Ekedahl, Vilma Helander, and Phuong T Nguyen. 2024. Programming with ChatGPT: How far can we go? *Machine Learning with Applications* 15 (2024), 100526.
- [12] Codeforces [n. d.]. Codeforces. <https://codeforces.com>. Accessed: 15-Aug-2023.
- [13] Carlos Eduardo Andino Coello, Mohammed Naze Alimam, and Rand Kouaty. 2024. Effectiveness of ChatGPT in coding: a comparative analysis of popular large language models. *Digital* 4, 1 (2024), 114–125.
- [14] Domenico Cotronco, Alessio Foggia, Cristina Improta, Pietro Liguori, and Roberto Natella. 2024. Automating the correctness assessment of AI-generated code for security contexts. *Journal of Systems and Software* (2024), 112113.
- [15] Jacob Devlin. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* (2018).
- [16] Sajib Acharjee Dip, Kazi Hasan Ibn Arif, Uddip Acharjee Shuvo, Ishtiaque Ahmed Khan, and Na Meng. 2024. Equitable Skin Disease Prediction Using Transfer Learning and Domain Adaptation. In *Proceedings of the AAAI Symposium Series*, Vol. 4. 259–266.
- [17] Sajib Acharjee Dip, Uddip Acharjee Shuvo, Tran Chau, Haoqiu Song, Petra Choi, Xuan Wang, and Liqing Zhang. 2024. PathoLM: Identifying pathogenicity from the DNA sequence through the Genome Foundation Model. *arXiv preprint arXiv:2406.13133* (2024).
- [18] Shihan Dou, Haoxiang Jia, Shenxi Wu, Huiyuan Zheng, Weikang Zhou, Muling Wu, Mingxu Chai, Jessica Fan, Caishuang Huang, Yunbo Tao, et al. 2024. What's Wrong with Your Code Generated by Large Language Models? An Extensive Study. *arXiv preprint arXiv:2407.06153* (2024).
- [19] Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M Zhang. 2023. Large language models for software engineering: Survey and open problems. In *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*. IEEE, 31–53.
- [20] Stefano Filippi and Barbara Motyl. 2024. Large Language Models (LLMs) in Engineering Education: A Systematic Review and Suggestions for Practical Adoption. *Information* 15, 6 (2024), 345.
- [21] HackerRank [n. d.]. HackerRank. <https://www.hackerrank.com>. Accessed: 15-Aug-2023.
- [22] Srinivasan Iyer, Ioannis Konstantas, Alvin Cheung, and Luke Zettlemoyer. 2018. Mapping language to code in programmatic context. *arXiv preprint arXiv:1808.09588* (2018).
- [23] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. 2024. A Survey on Large Language Models for Code Generation. *arXiv preprint arXiv:2406.00515* (2024).
- [24] Kailun Jin, Chung-Yu Wang, Hung Viet Pham, and Hadi Hemmati. 2024. Can ChatGPT Support Developers? An Empirical Evaluation of Large Language Models for Code Generation. In *2024 IEEE/ACM 21st International Conference on Mining Software Repositories (MSR)*. IEEE, 167–171.
- [25] Ranim Khojah, Mazen Mohamad, Philipp Leitner, and Francisco Gomes de Oliveira Neto. 2024. Beyond code generation: An observational study of chatgpt usage in software engineering practice. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 1819–1840.

- [26] Alexander LeClair, Sakib Haque, Lingfei Wu, and Collin McMillan. 2020. Improved code summarization via a graph neural network. In *Proceedings of the 28th international conference on program comprehension*. 184–195.
- [27] LeetCode [n. d.]. LeetCode. <https://leetcode.com>. Accessed: 15-Aug-2023.
- [28] Tianle Li, Ge Zhang, Quy Duc Do, Xiang Yue, and Wenhui Chen. 2024. Long-context llms struggle with long in-context learning. *arXiv preprint arXiv:2404.02060* (2024).
- [29] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2024. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *Advances in Neural Information Processing Systems* 36 (2024).
- [30] Stephen MacNeil, Andrew Tran, Juho Leinonen, Paul Denny, Joanne Kim, Arto Hellas, Seth Bernstein, and Sami Sarsa. 2022. Automatically generating cs learning materials with large language models. *arXiv preprint arXiv:2212.05113* (2022).
- [31] Tanha Miah and Hong Zhu. 2024. User Centric Evaluation of Code Generation Tools. In *2024 IEEE International Conference on Artificial Intelligence Testing (AITest)*. IEEE, 109–119.
- [32] Ansong Ni, Pengcheng Yin, Yilun Zhao, Martin Riddell, Troy Feng, Rui Shen, Stephen Yin, Ye Liu, Semih Yavuz, Caiming Xiong, et al. 2024. L2ceval: Evaluating language-to-code generation capabilities of large language models. *Transactions of the Association for Computational Linguistics* 12 (2024), 1311–1329.
- [33] Changan Niu, Ting Zhang, Chuanyi Li, Bin Luo, and Vincent Ng. 2024. On Evaluating the Efficiency of Source Code Generated by LLMs. In *Proceedings of the 2024 IEEE/ACM First International Conference on AI Foundation Models and Software Engineering*. 103–107.
- [34] Debalina Ghosh Paul, Hong Zhu, and Ian Bayley. 2024. Benchmarks and Metrics for Evaluations of Code Generation: A Critical Review. *arXiv preprint arXiv:2406.12655* (2024).
- [35] Sanka Rasnayaka, Guanlin Wang, Ridwan Shariffdeen, and Ganesh Neelakanta Iyer. 2024. An empirical study on usage and perceptions of llms in a software engineering project. In *Proceedings of the 1st International Workshop on Large Language Models for Code*. 111–118.
- [36] Mathieu Ravaut, Aixin Sun, Nancy Chen, and Shafiq Joty. 2024. On context utilization in summarization with large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2764–2781.
- [37] Steven I Ross, Fernando Martinez, Stephanie Houde, Michael Muller, and Justin D Weisz. 2023. The programmer's assistant: Conversational interaction with a large language model for software development. In *Proceedings of the 28th International Conference on Intelligent User Interfaces*. 491–514.
- [38] Zoltán Ságodi, István Siket, and Rudolf Ferenc. 2024. Methodology for Code Synthesis Evaluation of LLMs Presented by a Case Study of ChatGPT and Copilot. *IEEE Access* (2024).
- [39] Ensheng Shi, Fengji Zhang, Yanlin Wang, Bei Chen, Lun Du, Hongyu Zhang, Shi Han, Dongmei Zhang, and Hongbin Sun. 2023. SoTaNa: The Open-Source Software Development Assistant. *arXiv preprint arXiv:2308.13416* (2023).
- [40] Mohammed Latif Siddiq, Simantika Dristi, Joy Saha, and Joanna CS Santos. [n. d.]. The Fault in our Stars: Quality Assessment of Code Generation Benchmarks. [n. d.].
- [41] Mohammed Latif Siddiq, Lindsay Roney, Jiahao Zhang, and Joanna Cecilia Da Silva Santos. 2024. Quality Assessment of ChatGPT Generated Code and their Use by Developers. In *Proceedings of the 21st International Conference on Mining Software Repositories*. 152–156.
- [42] Alexey Svyatkovskiy, Shao Kun Deng, Shengyu Fu, and Neel Sundaresan. 2020. Intellicode compose: Code generation using transformer. In *Proceedings of the 28th ACM joint meeting on European software engineering conference and symposium on the foundations of software engineering*. 1433–1443.
- [43] Florian Tambon, Arghavan Moradi Dakhel, Amin Nikanjam, Foutse Khomh, Michel C Desmarais, and Giuliano Antoniol. 2024. Bugs in large language models generated code. *arXiv preprint arXiv:2403.08937* (2024).
- [44] Ben Arie Tanay, Lexy Arinze, Siddhant S Joshi, Kirsten A Davis, and James C Davis. 2024. An Exploratory Study on Upper-Level Computing Students' Use of Large Language Models as Tools in a Semester-Long Project. *arXiv preprint arXiv:2403.18679* (2024).
- [45] CodeGemma Team, Heri Zhao, Jeffrey Hui, Joshua Howland, Nam Nguyen, Siqi Zuo, Andrea Hu, Christopher A Choquette-Choo, Jingyue Shen, Joe Kelley, et al. 2024. Codegemma: Open code models based on gemma. *arXiv preprint arXiv:2406.11409* (2024).
- [46] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
- [47] UVa Online Judge [n. d.]. UVa Online Judge. <https://onlinejudge.org>. Accessed: 15-Aug-2023.
- [48] Jiayin Wang, Weizhi Ma, Peijie Sun, Min Zhang, and Jian-Yun Nie. 2024. Understanding User Experience in Large Language Model Interactions. *arXiv preprint arXiv:2401.08329* (2024).
- [49] Yanlin Wang, Tianyue Jiang, Mingwei Liu, Jiachi Chen, and Zibin Zheng. 2024. Beyond Functional Correctness: Investigating Coding Style Inconsistencies in Large Language Models. *arXiv preprint arXiv:2407.00456* (2024).
- [50] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
- [51] Pengcheng Yin and Graham Neubig. 2019. Reranking for neural semantic parsing. In *Proceedings of the 57th annual meeting of the association for computational linguistics*.
- [52] Daoguang Zan, Bei Chen, Fengji Zhang, Dianjie Lu, Bingchao Wu, Bei Guan, Yongji Wang, and Jian-Guang Lou. 2022. Large language models meet nl2code: A survey. *arXiv preprint arXiv:2212.09420* (2022).
- [53] Jingqing Zhang, Yao Zhao, Mohammad Saleh, and Peter Liu. 2020. Pegasus: Pre-training with extracted gap-sentences for abstractive summarization. In *International conference on machine learning*. PMLR, 11328–11339.
- [54] Pei Zhou, Jay Pujara, Xiang Ren, Xinyun Chen, Heng-Tze Cheng, Quoc V Le, Ed H Chi, Denny Zhou, Swaroop Mishra, and Huaixiu Steven Zheng. 2024. Self-discover: Large language models self-compose reasoning structures. *arXiv preprint arXiv:2402.03620* (2024).
- [55] Shuang Zhou, Zidu Xu, Mian Zhang, Chunpu Xu, Yawen Guo, Zaifu Zhan, Sirui Ding, Jiashuo Wang, Kaishuai Xu, Yi Fang, et al. 2024. Large language models for disease diagnosis: A scoping review. *arXiv preprint arXiv:2409.00097* (2024).